N-Axis Attribution OoD (2026-02-16)

2026-02-16

1　目的

本ドキュメントの目的は，**VLM における Attribution OoD の軸設計フェーズ**を，再現性・説明性・運用接続性の 3 観点で成立させることである．ここでの Attribution OoD とは，単に「分布外かどうか」を判定するのではなく，**どの種類の崩れが生じているか**を軸上で表現し，後段の判定器や運用アクションに接続する設計である．

本稿の研究範囲は，Step Separation の原則に従い，**最初 → 軸作成**までに限定する．したがって，後段のしきい値最適化や最終判定器の比較（AUROC 最大化）を同時に行わない．この切り分けにより，「軸が良いから性能が上がったのか」「判定器が良いから性能が上がったのか」を識別できる状態を確保する．

本稿が答えるべき研究問いは以下の 3 つである．

1. 軸数と正則化を，恣意的な重み関数なしで選べるか．
2. 得られた軸は seed 差分に対して安定か．
3. 得られた軸を，運用上意味のある 2 軸（z_u, z_c）へ固定できるか．

最終成果物は，固定軸の定義（数式），軸安定性評価，および可視化図である．これらを後段判定器フェーズの固定入力設計として用いる．

2　Attribution OoD の必要性 (現場背景)

従来の OoD 判定は，入力を ID/OoD の二値で扱う設計が中心であった．しかし，VLM は事前学習規模が大きく，「未知クラス」概念が従来より曖昧になるため，二値判定のみでは運用に必要な情報が不足する．

現場で本当に必要なのは，OoD 判定の**理由**である．同じ OoD でも，対処方法は大きく異なる．例えば，

1. 画像品質劣化（ブレ・ノイズ・低照度）なら再撮像で改善しやすい
2. 意味的不一致（未知対象・文脈不整合）なら人手確認が必要
3. 入力は明瞭だがモデル確信が低い場合は知識不足の可能性が高い

となる．

この差を判定系へ渡すため，本研究では **不確実性増大軸** z_u と **確信度低下軸** z_c を分離して扱う．これにより，運用アクションは次のように接続できる．

症状	軸空間での傾向	想定アクション
入力品質劣化	z_u 高，z_c 中〜低	再取得・再撮像・前処理改善
意味的不一致	z_c 高，z_u 中	人手介入・保留・ルール外処理
複合異常	z_u 高，z_c 高	強い警告と即時エスカレーション
通常入力	z_u 低，z_c 低	自動処理継続

Table 1: Attribution OoD の運用接続イメージ

すなわち，本研究の目的は「精度指標を 1 点上げること」ではなく，**誤りの原因を運用可能な座標系へ落とすこと**にある．

3 背景と問題点

本テーマの背景には，VLM 時代の OoD 検知における 3 つの構造的課題がある．

第 1 に，**軸数選択の恣意性**である．多くの実装では，性能・安定性・解釈性を混ぜた合成スコアを使うが，係数の選び方次第で最適解が変わる．これは研究者自由度を増やし，再現性を下げる．

第 2 に，**小規模評価の不安定性**である．サンプル数が小さいと，ロングテールの失敗パターンを十分に観測できない．結果として，特定分割に依存した軸が選ばれやすくなる．

第 3 に，**軸定義の手動上書き**である．SparsePCA で軸候補を得ても，最終的に手作業で軸式を作り直すと，「データ駆動で軸を得た」という主張が弱くなる．

本研究では，これらを以下の方針で回避する．

1. データ規模を ID/OoD 合計 20,000 まで拡大
2. 分散閾値で定数特徴を除去し，数値不安定を先に抑止
3. SparsePCA の CV 再構成誤差に基づき 1SE ルールで選択
4. 軸学習結果をそのまま固定し，後段と分離

この設計により，「軸を先に固定してから判定器を評価する」実験秩序を維持できる．

4 解決方針 (何を解くか)

解く問題を次のように明確化する．

入力: VLM の推論ログから作る差分特徴 $\mathbf{x}_i$.

出力: 再現可能に固定された軸群 $\{z_j\}_{j=1}^{k}$ と，運用用 2 軸 (z_u, z_c) への割当.

制約: 軸選択は統計的手続き（CV+1SE）で行い，最終判定器情報を使わない（情報リーク禁止）.

実装上の意思決定は以下である．

1. 軸抽出法は SparsePCA を採用（線形・スパース・可読性重視）
2. 軸数 k と正則化 α はグリッド探索
3. 評価指標は再構成誤差 MSE（CV 平均と標準誤差）
4. 1SE ルールで最小複雑モデルを採用

この方針の利点は，「**高性能そうに見える設定**」ではなく「**再現可能で説明可能な設定**」を機械的に選べる点である．

5 実行環境

項目	内容
実行基盤	Google Colab (Python 3.12)
GPU	NVIDIA A100-SXM4-40GB
PyTorch	2.9.0+cu128
主要ライブラリ	open-clip-torch 2.26.1, scikit-learn 1.5.2, pandas 2.2.x
VLM backend	OpenCLIP ViT-B-32 (laion2b_s34b_b79k)
ID dataset	TFDS food101 train[:10000]
OoD dataset 1	TFDS cifar100 train[:5000]
OoD dataset 2	TFDS imagenet_r test[:5000]
総サンプル数	20,000
出力成果物	軸定義，安定性評価，可視化図

Table 2: 軸作成実験の環境設定

補足として，初回実行時の TFDS ダウンロード時間は長くなるが，再実行時はローカルキャッシュを再利用するため高速化される．また，CUDA 系の警告（cuDNN/cuBLAS 重複登録）は環境由来であり，本実験の失敗原因ではなかった．

6 処理フロー

軸作成の手順を，入出力を含めて具体化する．

1. **データ取得**
   TFDS から ID/OoD を所定件数読み込む．
2. **VLM 推論**
   OpenCLIP で各サンプルのクラス分布を計算し，conf，entropy，energy，ood_score を記録する．
3. **差分特徴構成**
   ID 平均を基準に，drop/gain 特徴を作る（確信度低下，不確実性増大など）．
4. **前処理**
   Variance Threshold で定数特徴を削除し，何を落としたかを実験ログとして記録する．
5. **モデル選択**
   候補 (k, α) で CV 再構成誤差を計算し，1SE 選択用の比較表を得る．
6. **1SE 選択**
   最小 MSE と標準誤差から閾値を作り，その範囲で最小複雑モデルを選ぶ．
7. **最終軸固定**
   選択された (k, α) で全データ再学習し，軸荷重と軸スコアを固定する．
8. **運用軸割当**
   不確実性寄与が最大の軸を z_u，確信度寄与が最大の軸を z_c として固定する．

このフローにより，後段判定器の入力は完全に固定される．

7　数理定義

サンプル指標

サンプル i の予測確率分布を $\mathbf{p}_i = (p_{i1}, \ldots, p_{iC})$ とする．最大確率を

$$\mathrm{conf}_i = \max_j p_{ij} \tag{1}$$

と定義し，エントロピーを

$$H_i = -\sum_{j=1}^{C} p_{ij} \log(p_{ij} + \epsilon) \tag{2}$$

とする（$\epsilon > 0$ は数値安定化）．

クラス数依存を除くため，正規化エントロピー

$$\tilde{H}_i = \frac{H_i}{\log(\max(2, C))} \tag{3}$$

を用いる．複合 OoD スコアは

$$s_i^{\mathrm{ood}} = 0.5(1 - \mathrm{conf}_i) + 0.5\tilde{H}_i \tag{4}$$

とする．

差分特徴

ID 集合を $\mathcal{D}_{\mathrm{ID}}$ とし，その平均を $\bar{m}_{\mathrm{ID}}$ で表す．例えば

$$d_{\mathrm{conf_drop},i} = \bar{\mathrm{conf}}_{\mathrm{ID}} - \mathrm{conf}_i, \tag{5}$$

$$d_{\mathrm{entropy_gain},i} = \tilde{H}_i - \bar{\tilde{H}}_{\mathrm{ID}}, \tag{6}$$

$$d_{\mathrm{oodscore_gain},i} = s_i^{\mathrm{ood}} - \bar{s}_{\mathrm{ID}}^{\mathrm{ood}} \tag{7}$$

を定義する．同様に $d_{\mathrm{msp_drop}}, d_{\mathrm{energy_gain}}, d_{\mathrm{abstain_gain}}$ を作り，

$$\mathbf{x}_i = [d_{\mathrm{conf_drop}}, d_{\mathrm{msp_drop}}, d_{\mathrm{entropy_gain}}, d_{\mathrm{energy_gain}}, d_{\mathrm{oodscore_gain}}, d_{\mathrm{abstain_gain}}]^\top \tag{8}$$

を軸学習の入力とする．

分散閾値前処理

特徴 x_j の標本分散を $\mathrm{Var}(x_j)$ とし，

$$\mathrm{Var}(x_j) < \tau_{\mathrm{var}} \tag{9}$$

を満たす特徴を削除する（本実験では $\tau_{\mathrm{var}} = 10^{-10}$）．これにより，定数特徴由来の数値不安定を防ぐ．

SparsePCA

標準化後行列を $X \in \mathbb{R}^{n \times d}$ とする．SparsePCA は，成分行列 W に L1 型のスパース性を課し，再構成誤差を最小化する方向を求める．本実装では scikit-learn の `SparsePCA` を用い，候補 $k \in \{1, 2, 3, 4\}$，$\alpha \in \{0.5, 1.0, 2.0, 4.0, 8.0\}$ を探索した．

CV 再構成誤差と 1SE ルール

fold f における検証誤差を

$$\mathrm{MSE}_f(k,\alpha) = \frac{1}{|\mathcal{V}_f|}\sum_{i\in\mathcal{V}_f}\|\mathbf{x}_i - \hat{\mathbf{x}}_i^{(k,\alpha)}\|_2^2 \tag{10}$$

とし，

$$\overline{\mathrm{MSE}}(k,\alpha) = \frac{1}{F}\sum_{f=1}^{F}\mathrm{MSE}_f(k,\alpha), \quad \mathrm{SE}(k,\alpha) = \frac{\mathrm{Std}(\mathrm{MSE}_f)}{\sqrt{F}} \tag{11}$$

を計算する．

最小誤差を与える候補を (k^*,α^*) とし，

$$\tau_{\mathrm{1SE}} = \overline{\mathrm{MSE}}(k^*,\alpha^*) + \mathrm{SE}(k^*,\alpha^*) \tag{12}$$

を定義する．採用候補集合

$$\mathcal{C} = \{(k,\alpha) \mid \overline{\mathrm{MSE}}(k,\alpha) \le \tau_{\mathrm{1SE}}\} \tag{13}$$

から，最小の k（同率なら大きい α）を選ぶ．

最終固定軸

本実験で選択された設定は

$$k = 4, \quad \alpha = 8.0 \tag{14}$$

である．そのとき，運用軸として割り当てた式は

$$z_u = +0.9187\, d_{\text{entropy_gain}} + 0.3949\, d_{\text{oodscore_gain}} \tag{15}$$

$$z_c = +0.6546\, d_{\text{conf_drop}} + 0.6546\, d_{\text{msp_drop}} + 0.3783\, d_{\text{oodscore_gain}} \tag{16}$$

である．実装上は msp = conf であるため，

$$z_c \simeq +1.3092\, d_{\text{conf_drop}} + 0.3783\, d_{\text{oodscore_gain}} \tag{17}$$

と等価整理できる．

8 結果

基礎統計

ID 基準平均は

$$\bar{\mathrm{conf}}_{\mathrm{ID}} = 0.8259888770, \quad \bar{\tilde{H}}_{\mathrm{ID}} = 0.1187297247, \quad \bar{s}_{\mathrm{ID}}^{\mathrm{ood}} = 0.1463704238 \tag{18}$$

であった．

1SE 選択結果

項目	値	解釈
k_{selected}	4	4 軸で十分な再構成
α_{selected}	8.0	強めのスパース化
MSE_{min}	8.3656×10^{-5}	最小 CV 誤差
SE_{min}	4.7763×10^{-7}	誤差の推定ばらつき
τ_{1SE}	8.4134×10^{-5}	許容閾値

Table 3: 1SE ルールによる選択結果

seed 安定性

seed 42/43/44 で再実行した結果，軸方向のコサイン類似度（絶対値）はほぼ 1.0 であった．

seed	k	α	axis_u	axis_c	$\lvert\cos\rvert$（seed42 基準）
42	4	8.0	3	0	$(z_u, z_c) = (1.000000, 1.000000)$
43	4	4.0	3	0	$(z_u, z_c) = (0.999998, 0.999999)$
44	4	8.0	3	0	$(z_u, z_c) = (1.000000, 1.000000)$
平均	-	-	-	-	$(0.99999948, 0.99999983)$

Table 4: 3-seed での軸安定性

この結果は，**軸の向きが乱数にほとんど依存しない**ことを示す．後段で判定器のみを比較する条件として十分である．

図による検証結果

図
reffig:scatter は，z_u-z_c 空間での ID/OoD 分布を直接示す．図
reffig:density は，散布図の見かけ密度を補う等高線可視化であり，両者の重なり領域と分離方向を明確にする．図
reffig:loadings は，各軸の特徴寄与を定量的に示し，式で定義した z_u, z_c の意味づけを支える．図
reffig:cv は，CV 誤差曲線と 1SE 閾値を重ね，(k, α) 選択が再現可能な手続きであることを可視化する．

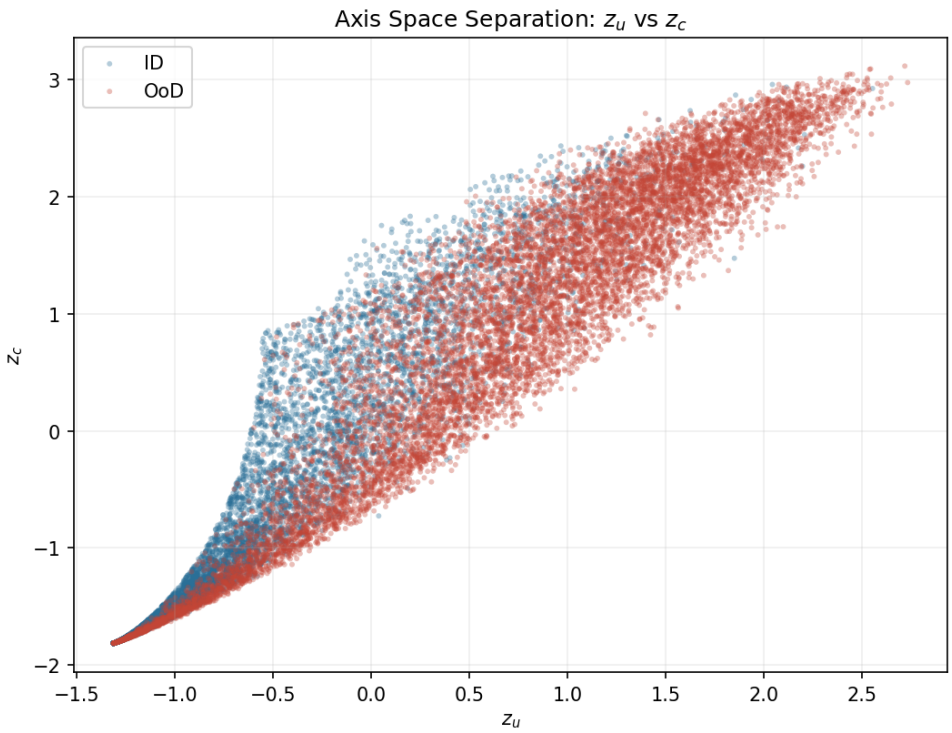

Figure 1: z_u-z_c 散布図（ID/OoD 色分け）

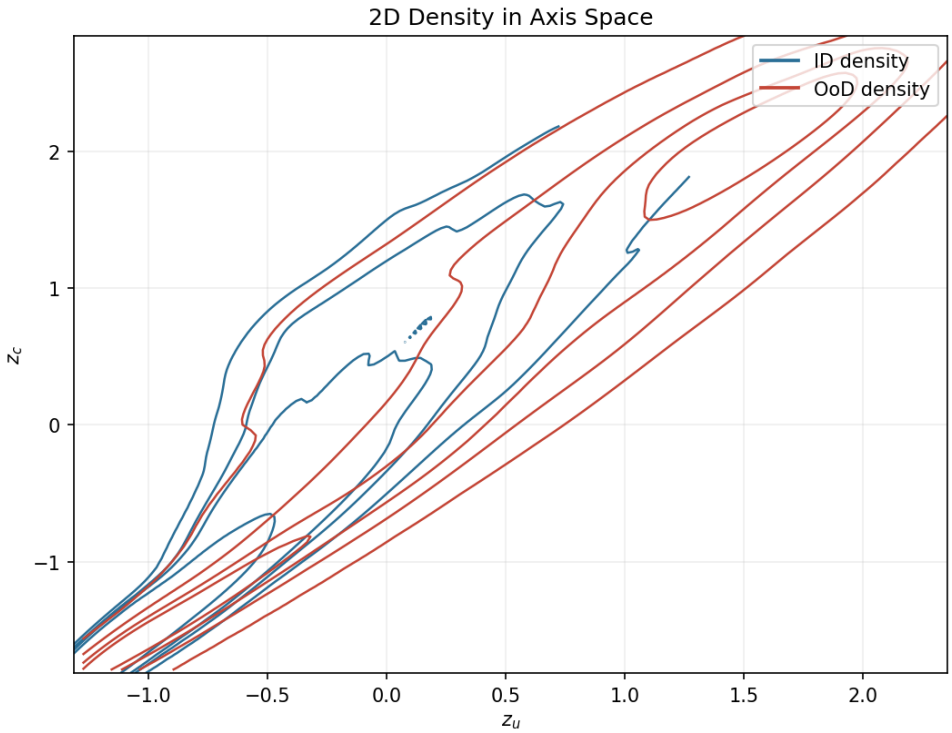

Figure 2: z_u-z_c 密度等高線（ID/OoD）

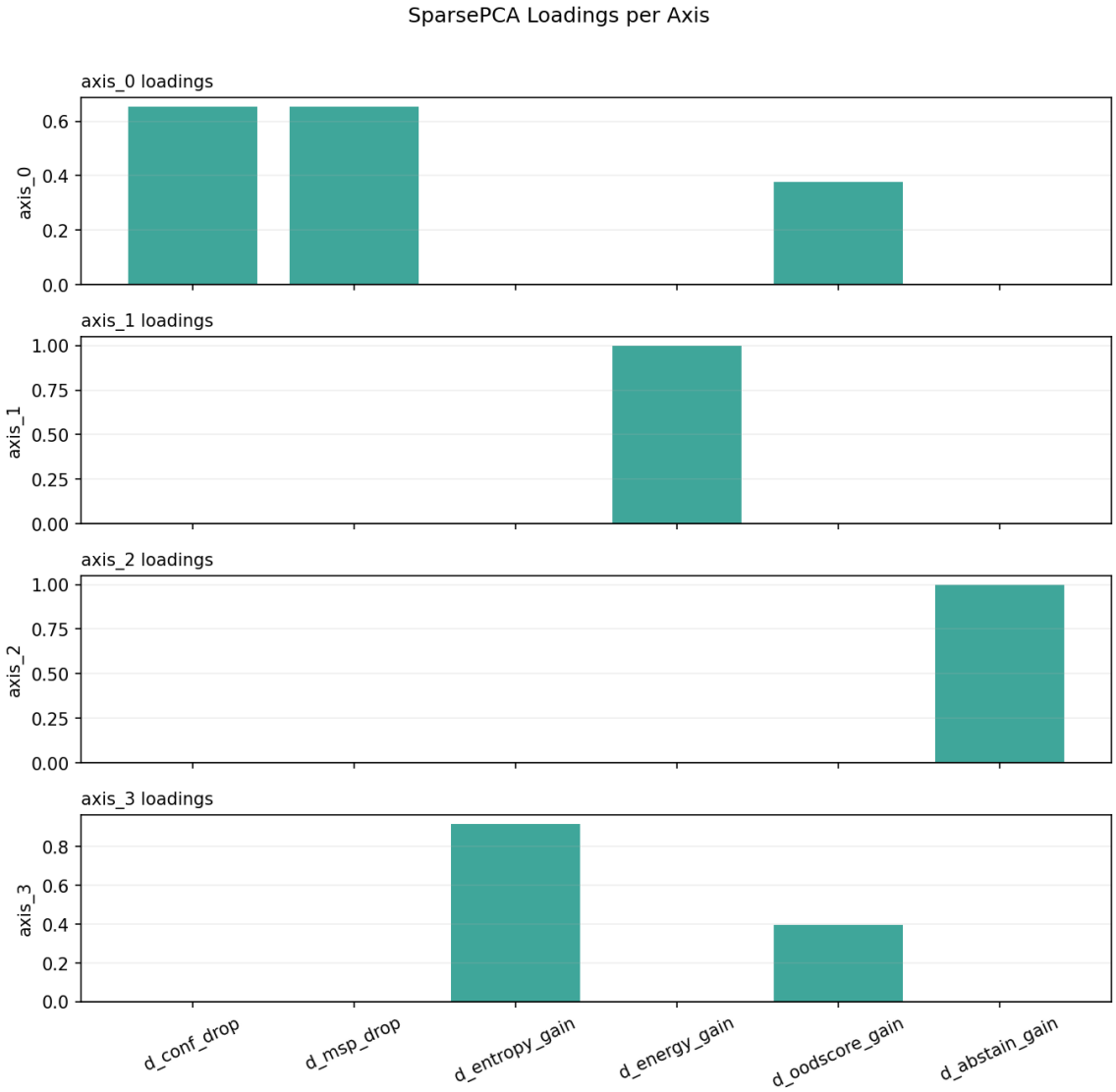

Figure 3: SparsePCA 軸荷重（axis loadings）

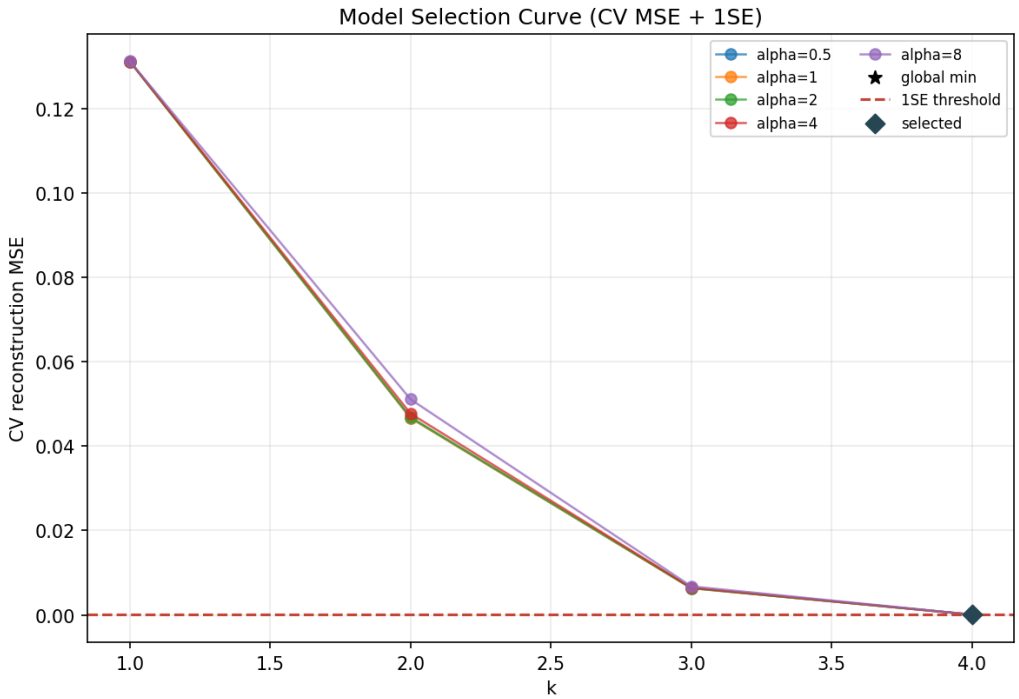

Figure 4: CV 再構成誤差と 1SE 選択

9 可視化 (グラフ)

軸作成フェーズで必須の可視化は次の 4 種類である．

1. **z_u-z_c 散布図（ID/OoD 色分け）**
   ID と OoD がどの方向で分離するかを視覚確認する．
2. **2D 密度等高線（KDE）**
   散布図だけでは見えにくい密度ピークと重なり領域を確認する．
3. **軸 loadings 棒グラフ**
   各軸がどの特徴で構成されるかを定量確認する．
4. **CV 誤差曲線（1SE 閾値付き）**
   選択した (k, α) の妥当性を説明する基礎図となる．

本稿では，これら 4 種類を結果章に直接挿入し，「数値表のみの主張」ではなく「視覚情報を伴う主張」に拡張した．

10 考察

本結果から得られる示唆は 3 点ある．

第 1 に，1SE ルールの導入により，軸数選択をヒューリスティック重み関数から切り離せた．これは論文査読で問われる「なぜその軸数か」に対し，手続き的かつ再現可能な回答を与える．

第 2 に，seed 安定性が非常に高いことから，データ規模 20,000 は軸固定フェーズとして妥当である．この点は，先行の小規模検証で問題だった揺らぎを実質的に解消している．

第 3 に，z_c へ $d_{\text{oodscore_gain}}$ が寄与している点は重要である．これは現実の失敗モードが「純粋な確信度低下」だけでなく，不確実性要素を伴う混合モードである可能性を示す．言い換えると，運用上は軸の完全直交を仮定しない方が安全である．

一方で制約も残る．

1. OoD 集合が自然画像寄り（cifar100, imagenet_r）であり，ドメイン外シフトの種類は限定的である．
2. プロンプト形式は固定であり，VLM 特有のプロンプト感度は未評価である．
3. 軸は線形結合であり，非線形失敗モードを直接は表現しない．

したがって次工程では，軸固定を維持したまま，判定器側で非線形境界や条件別評価を導入するのが妥当である．

11 結論

本稿では，研究計画の前半（最初 → 軸作成）を完了し，以下を達成した．

1. ID/OoD 合計 20,000 サンプルの軸作成基盤を構築
2. 分散閾値 + SparsePCA + 1SE で恣意性の低い選択を実現
3. $k = 4, \alpha = 8.0$ を選択し，運用軸 z_u, z_c を確定
4. 3-seed でほぼ完全一致の軸安定性を確認

次の工程は，固定軸を入力にした判定器設計である．このときも Step Separation を維持し，判定器比較（AUROC/AUPR/FPR95，95% 信頼区間）を軸の再学習なしで実施する．

以上より，本研究は「理由付き OoD 検知」の前提条件である軸定義の再現可能化を達成した．